

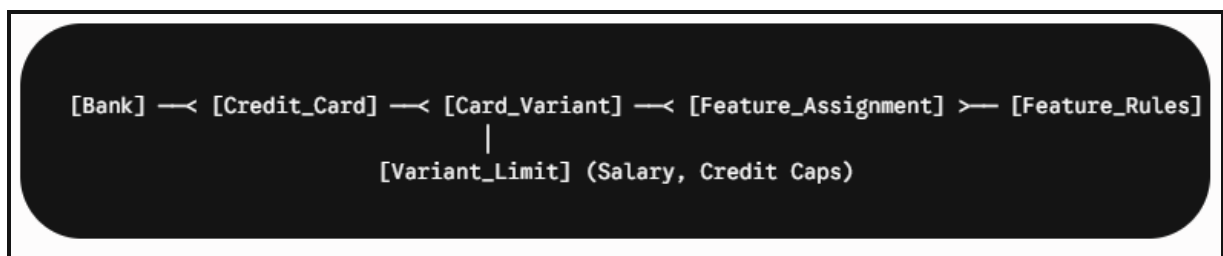
To design a highly generalized, robust data model for United Arab Emirates (UAE) credit cards, we have to look past the standard marketing text. In the UAE market, cards are intensely hyper-segmented. You have cards offering **10% cashback on UnionCoop/Carrefour**, **5% on fuel at ADNOC/Emarat**, **BOGO movie tickets at VOX/Reel/Novo**, **4 free airport lounge visits via LoungeKey/Priority Pass**, and **0% Easy Payment Plans (EPP) at Sharaf DG**.

If you hardcode these categories as table columns (e.g., `cashback_groceries`, `lounge_access`), your schema will quickly fall apart the next time a bank launches a custom campaign (like FAB's step-tracking lifestyle rewards or valet parking features).

To solve this, we will use a **Polymorphic Rule-Based Configuration Model**. Instead of treating an "offer" or a "limit" as text, we break them down into **Triggers (Conditions)** and **Actions (Outcomes)**.



The Core Entity-Relationship Diagram (Conceptual)



The Generalized SQL Schema

This PostgreSQL schema dynamically maps complex logic such as *"If Merchant Category Code (MCC) is 5411 (Groceries) and Spend > 500 AED, then give 5% Cashback up to 200 AED/month"*.

SQL

```
-- 1. Lookups & Meta Tables
```

```
CREATE TABLE banks
```

```
  bank_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
```

```
  bank_name VARCHAR 100 NOT NULL -- e.g., 'Emirates NBD', 'FAB', 'ADCB'
```

```
  country_code VARCHAR 3 DEFAULT 'ARE'
```

```
;
```

```
CREATE TABLE card_networks
```

```
  network_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
```

```
  network_name VARCHAR 50 NOT NULL -- 'Visa', 'Mastercard', 'Amex'
```

```

CREATE TABLE card_tiers
  tier_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  tier_name VARCHAR 50 NOT NULL -- 'Platinum', 'Signature', 'Infinite', 'World Elite'

```

-- 2. Core Card Tables

```

CREATE TABLE credit_cards
  card_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  bank_id INT REFERENCES banks(bank_id)
  card_name VARCHAR 150 NOT NULL -- e.g., 'Skywards Infinite', '365 Cashback'
  is_active BOOLEAN DEFAULT TRUE

```

```

CREATE TABLE card_variants
  variant_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  card_id INT REFERENCES credit_cards(card_id)
  network_id INT REFERENCES card_networks(network_id)
  tier_id INT REFERENCES card_tiers(tier_id)
  annual_fee_primary NUMERIC 10 2 NOT NULL
  annual_fee_supplemental NUMERIC 10 2 DEFAULT 0.00
  min_salary_requirement NUMERIC 10 2 -- Highly critical for UAE market onboarding
  currency VARCHAR 3 DEFAULT 'AED'

```

-- 3. Dynamic Eligibility & Limits Engine

-- Handles things like minimum/maximum caps, salary caps, and processing variables

```

CREATE TABLE variant_limits
  limit_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  variant_id INT REFERENCES card_variants(variant_id)
  limit_type VARCHAR 50 NOT NULL -- 'MIN_SALARY', 'MAX_CREDIT_LIMIT',
'CASH_ADVANCE_LIMIT'
  limit_value NUMERIC 12 2 NOT NULL
  conditions json JSONB -- For corner-case rules

```

-- 4. Highly Generalized Feature & Offer Engine

```

CREATE TABLE features_and_offers
  feature_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  feature_type VARCHAR 50 NOT NULL -- 'CASHBACK', 'MILES', 'LOUNGE', 'EPP', 'CINEMA',
'VALET'
  title VARCHAR 255 NOT NULL -- e.g., 'VOX Cinema Buy 1 Get 1'

```

```

provider_name VARCHAR 100 -- e.g., 'VOX Cinemas', 'ADNOC', 'Majid Al Futtaim'
start_date DATE
end_date DATE -- Null means perpetual card benefit
is_promotional BOOLEAN DEFAULT FALSE

```

```

-- The Bridge Table mapping features to specific card variants
CREATE TABLE card_feature_assignments
(
  assignment_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  variant_id INT REFERENCES card_variants(variant_id)
  feature_id INT REFERENCES features_and_offers(feature_id)
  is_opt_in_required BOOLEAN DEFAULT FALSE -- Some UAE apps require you to 'Activate' offers
)

```

```

-- 5. The Brains: Rule & Condition Logic (JSONB Powered)
CREATE TABLE feature_rules
(
  rule_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY
  feature_id INT REFERENCES features_and_offers(feature_id)
  -- JSON structure to capture spending boundaries dynamically
  -- e.g., {"mcc": ["5411", "5499"], "spend_currency": "AED", "min_transaction": 100}
  trigger_conditions JSONB NOT NULL
  -- JSON structure to map the calculation result
  -- e.g., {"reward_type": "PERCENT", "value": 5.00, "monthly_cap": 300}
  outcome_action JSONB NOT NULL
)

```

Out-of-the-Box JSONB Representation For Complex Logic

Using standard columns forces structural migrations anytime a bank invents a new quirk. Using JSONB indexes ensures you can query inside structural fields without losing performance.

Example 1: Tiered Cashback (e.g., *ADCB 365* or *Liv Cashback+*)

Rule: 6% Cashback on Dining, but capped at 200 AED a month.

- **trigger_conditions:**

```

JSON
{
  "mcc_groups" "DINING" "RESTAURANTS"
  "channel" "ALL"
  "geography" "DOMESTIC"
}

```

- **outcome_action:**

```

JSON
{
  "benefit_type": "CASHBACK",
  "calculation": "PERCENTAGE",
  "rate": 0.06,
  "capping": {
    "limit": 200.00,
    "period": "MONTHLY"
  }
}

```

Example 2: Lifestyle Perks (e.g., VOX Movie Tickets Buy-1-Get-1)

Rule: Buy 1 get 1 free at VOX Cinemas, maximum 4 free tickets per month, provided you spend 2,000 AED overall on the card in the previous month.

- **trigger_conditions:**

```

JSON
{
  "merchant_identity": "VOX_CINEMAS_UAE",
  "minimum_previous_cycle_spend": 2000.00
}

```

- **outcome_action:**

```

JSON
{
  "benefit_type": "BOGO",
  "max_free_units": 4,
  "period": "MONTHLY"
}

```

Access Layer: Common Queries for UAE Scenarios

Because this model stores conditions in JSONB, we can utilize index searches (@>) to pull back complex calculations instantly.

Query 1: Find cards that offer rewards on Groceries (MCC 5411) in the UAE

```

SQL
SELECT
    b.bank_name
    cc.card_name
    fo.title
    fro.outcome_action -->'rate' AS cashback_rate
FROM card_variants cv
JOIN credit_cards cc ON cv.card_id = cc.card_id
JOIN banks b ON cc.bank_id = b.bank_id
JOIN card_feature_assignments cfa ON cv.variant_id = cfa.variant_id
JOIN features_and_offers fo ON cfa.feature_id = fo.feature_id
JOIN feature_rules fr ON fo.feature_id = fr.feature_id
WHERE fo.feature_type = 'CASHBACK'
-- Checks if '5411' exists inside the JSON array trigger_conditions -> mcc
AND fr.trigger_conditions ->'mcc' @> '"5411"'

```

Query 2: Filter cards for an applicant making 15,000 AED/month looking for Lounge Access

```

SQL
SELECT
    b.bank_name
    cc.card_name
    cv.annual_fee_primary
    fo.title AS lounge_benefit
FROM card_variants cv
JOIN credit_cards cc ON cv.card_id = cc.card_id
JOIN banks b ON cc.bank_id = b.bank_id
JOIN card_feature_assignments cfa ON cv.variant_id = cfa.variant_id
JOIN features_and_offers fo ON cfa.feature_id = fo.feature_id
WHERE cv.min_salary_requirement <= 15000
AND fo.feature_type = 'LOUNGE'

```

Key Advantages of this Model

1. **No Schema Changes on New Offers:** If a bank launches a promotional discount with

Talabat or *Careem Plus* for 3 months, you simply write a new row to `features_and_offers` and `feature_rules` with an `end_date`.

2. **Deterministic Transaction Engine:** When an authorization request comes through, your application can parse the transaction info (MCC, Amount, Merchant Name), pass it through a validation matcher against `trigger_conditions`, and instantly present the client with the calculated reward.
3. **Multi-Bank Friendly:** The high isolation abstraction (`banks` $\rightarrow$ `credit_cards` $\rightarrow$ `card_variants`) ensures you can easily keep multi-tenant or multi-bank data perfectly siloed yet structured identically.